Project Report
on

SportTrackOTM – Online Sports Team Management System

Submitted by

Dhiraj Bhagade Bhalchandra

Under the Internal Guidance of
Prof. Jaydeep Lokhande

Department of Computer Engineering, Flora Education Society's
Flora Institute of Technology,
Pune
Academic Year 2026–2027

DEPARTMENT OF COMPUTER ENGINEERING

C E R T I F I C A T E

This is to certify that, **Dhiraj Bhagade Bhalchandra** of T.E. (Computer Engineering) has completed the project **SportTrackOTM – Online Sports Team Management System** and has submitted this TECHNICAL SEMINAR Report towards fulfillment of the requirement for the Subject Technical Seminar – [Subject Code] for the academic year 2026-2027.

Prof. Jaydeep Lokhande
Project Guide
Department of Computer Engineering
FIT, Pune

Prof. Jaydeep Lokhande
Head of Department
Department of Computer Engineering
FIT, Pune

Internal Examiner: _____

External Examiner: _____

Date: _____

Acknowledgement

I would like to express my sincere gratitude to my project guide, Prof. Jaydeep Lokhande, Department of Computer Engineering, Flora Institute of Technology, Pune, for his valuable guidance, constant encouragement and insightful suggestions during the planning of SportTrackOTM.

I am thankful to the Head of the Department of Computer Engineering for providing the academic environment and facilities required for this work. I also extend my thanks to the Principal and the management of Flora Education Society's Flora Institute of Technology, Pune, for their support.

I am grateful to all the teaching and non-teaching staff of the department for their help during the preparation of this report. I also acknowledge the authors and maintainers of the technical documentation and research resources referred to while studying web application development, database design and sports management systems.

Finally, I thank my family and friends for their encouragement, patience and moral support.

Dhiraj Bhalchandra Bhagade

Abstract

Sports teams commonly coordinate player information, practice sessions, match schedules, attendance, performance records and announcements through paper registers, spreadsheets and separate messaging applications. These disconnected methods lead to duplicated information, missed updates, difficulty in maintaining historical records and limited visibility into team and player progress. SportTrackOTM (Online Sports Team Management System) is proposed as a responsive web application that centralizes these activities in a single platform.

The system is planned around three roles: administrator, coach and player. Administrators manage users and teams; coaches organise team activities, schedule matches and training sessions, record attendance, enter results and maintain player performance; players view their profiles, schedules, attendance, performance summaries, results and announcements. Rolebased access restricts each function to the users responsible for it, and permissions are enforced on the server.

The implementation uses HTML, CSS, Bootstrap and JavaScript for the user interface, Python with Django for server-side logic, and MySQL for persistent storage. Django's authentication, URL routing, forms and ORM support a secure and maintainable design. The report presents the problem definition, literature survey, requirements, architecture, database design, implementation approach, test plan, applications and future enhancements. As development is at the planning and setup stage, no performance results are claimed; the report instead provides a test framework to be filled with actual results after implementation.

Keywords: *Sports Team Management, Player Management, Match Scheduling, Attendance Tracking, Performance Monitoring, Role-Based Access Control, Django, MySQL.*

List of Figures

Figure 5.1: High-level architecture of SportTrackOTM (proposed). 9
Figure 5.2: Use-case view of the role-based system. 10
Figure 5.3: Data flow diagram of SportTrackOTM. 11
Figure 5.4: Proposed entity-relationship overview. 12
Figure 5.5: Sequence of a protected request. 13
Figure 6.1: Proposed project schedule (16 weeks). 14

List of Tables

Table 2.1: Comparison of existing and proposed approaches 4
Table 3.1: Challenges in team administration and the proposed solutions 5
Table 4.1: Hardware requirements 7
Table 4.2: Software requirements 7
Table 4.3: Functional requirements 7
Table 4.4: Non-functional requirements 8
Table 5.1: Role-wise permission matrix (planned) 10
Table 5.2: Main system modules 11
Table 5.3: Proposed database entities 12
Table 6.1: Project phases, activities and deliverables 14
Table 6.2: Project risks and mitigation 15
Table 8.1: Recommended test conditions 20
Table 8.2: Functional test cases 20
Table 9.1: Result recording template 22
Table 10.1: Application areas of SportTrackOTM 24
Table 12.1: Future work directions 26

Contents

Abstract **I**

List of FiguresII **II**

List of Tables **III**

Chapter 1: Introduction **1**

1.1 Background 1

1.2 Motivation 1

1.3 Overview of the Proposed System 1

1.4 Objectives 2

1.5 Scope of the Project 2

1.6 Organization of the Report 2

Chapter 2: Literature Survey **3**

2.1 Existing Practices 3

2.2 Digital Sports Management 3

2.3 Web Application Technologies 3

2.4 Research Gap 3

Chapter 3: Problem Statement **5**

3.1 Problem Definition 5

3.2 Challenges and Proposed Solutions 5

3.3 Assumptions and Constraints 5

3.4 Expected Outcomes 5

Chapter 4: Project Requirements **6**

4.1 Hardware Requirements7 6

4.2 Software Requirements 6

4.3 Functional Requirements 6

4.4 Non-Functional Requirements 7

4.5 Feasibility Study 7

Chapter 5: System Analysis – Proposed Architecture / High-Level Design95.1 Overall Architecture **8**

5.2 Role-Based Workflow 8

5.3 Data Flow 9

5.4 Module Description 10

5.5 Database Design 10

5.6 Sequence of Operation 12

5.7 Dashboard Design 12

5.8 Design Considerations 12

Chapter 6: Project Plan **13**

6.1 Project Phases 13

6.2 Risk Analysis 13

6.3 Resources and Tools 14

Chapter 7: Implementation **15**

7.1 Implementation Environment 15
7.2 Project Structure 15
7.3 Database Configuration 15
7.4 Authentication and Roles 16
7.5 Team and Player Management 16
7.6 Match and Training Management 17
7.7 Attendance and Performance 17
7.8 Announcements and Reports 18
Chapter 8: Performance Evaluation and Testing 19
8.1 Evaluation Strategy 19
8.2 Evaluation Metrics 19
8.3 Test Conditions20 19
8.4 Functional Test Plan 19
8.5 Non-Functional Testing 20
Chapter 9: Results and Analysis 21
9.1 Current Development Status 21
9.2 Result Table 21
9.3 Analysis Framework 21
9.4 Discussion 21
Chapter 10: Applications 22
Chapter 11: Conclusions 23
Chapter 12: Future Prospects of the Project26 24
References 25

Chapter 1 Introduction

1.1 Background

Sports teams, whether at a college, club or academy level, depend on a steady flow of information: who is on the roster, which coach is responsible for which team, when and where the next match or practice session takes place, who attended the last session, and how each player has performed. In many teams these tasks are handled with paper registers, spreadsheets and messaging groups. Each tool solves a small part of the problem, but none of them gives a single, reliable view of the team.

The consequences are familiar. A change of venue announced in a chat group is missed by half the squad. Two versions of the same spreadsheet disagree about attendance. A coach who wants to review a player's progress over a season must search through old files. As the number of players, teams and fixtures grows, this informal approach becomes slow, error-prone and difficult to audit.

A centralized web application addresses these problems by keeping all team data in one relational database and by showing each user only the information and actions that suit their role. Because it runs in a browser, it can be used from a laptop in the sports office or from a phone on the ground, without installing anything.

1.2 Motivation

The motivation for **SportTrackOTM** is to reduce repetitive administrative work and to improve coordination between administrators, coaches and players. The project is also a practical opportunity to apply core computer-engineering concepts: relational database design, server-side web development, authentication and authorization, form validation and software testing.

A browser-based system with a responsive interface was chosen over a native mobile application because it works on any device with a modern browser, is easier to maintain, and can be developed within the time available for a student project.

1.3 Overview of the Proposed System

SportTrackOTM (Online Sports Team Management System) is a role-based web application with three kinds of users:

- **Administrator** – creates and manages user accounts, teams, coaches and players, and can publish general announcements.
- **Coach** – manages the assigned team: schedules matches and training sessions, records attendance, enters match results and player performance, and posts team announcements.
- **Player** – views his or her own profile, team, schedule, attendance, performance, results and announcements.

The user interface is built with HTML, CSS, Bootstrap and JavaScript. Server-side logic, authentication and permission checks are handled by Python with the Django framework, and

all persistent data is stored in a MySQL database. After login, every user lands on a dashboard that summarises what matters most for that role.

1.4 Objectives

- To provide a centralized platform for team, player and coach records.
- To allow coaches to schedule matches and training sessions.
- To maintain attendance records for training sessions.
- To store match results and player performance information.
- To communicate announcements and schedule changes through a single system.
- To apply role-based access control so that users see only the functions they are permitted to use.
- To create a responsive interface suitable for desktop, tablet and mobile browsers.
- To define a clear test plan so that each feature can be verified and its result recorded.

1.5 Scope of the Project

The first version covers user authentication, role-based dashboards, team and player management, match scheduling, training scheduling, attendance, performance records, match results, announcements and basic reports.

The following are outside the scope of the first version and are discussed as future prospects in Chapter 12: online payments, live score feeds, GPS or wearable tracking, automated team selection, and a native mobile application.

1.6 Organization of the Report

Chapter 2 reviews existing practices and related technologies and identifies the research gap. Chapter 3 states the problem. Chapter 4 lists hardware, software, functional and non-functional requirements and the feasibility study. Chapter 5 presents the system analysis and proposed architecture. Chapter 6 gives the project plan. Chapter 7 explains the implementation approach. Chapter 8 describes performance evaluation and testing, and Chapter 9 provides the status and a framework for recording results. Chapters 10, 11 and 12 cover applications, conclusions and future prospects, followed by the references.

Chapter 2 Literature Survey

2.1 Existing Practices

Small and amateur sports teams most commonly rely on paper registers, spreadsheets, messaging applications and shared calendars. These tools are familiar and inexpensive, and that is why they remain popular. Their weakness is fragmentation: information lives in several places, there is no single owner of the data, and nothing prevents a player from accidentally editing a shared file. A change in match venue may not reach every player, and historical attendance or performance records are difficult to compare across a season.

2.2 Digital Sports Management

Commercial team-management platforms generally combine roster administration, scheduling, attendance, communication and performance records. Their common design principle is to keep structured information in one place and to give users access according to their responsibilities. These products are feature-rich but often subscription-based, and they may include far more functionality than a college or local club needs. SportTrackOTM adopts the same centralized approach, but with a deliberately limited and understandable feature set that suits an academic project.

2.3 Web Application Technologies

HTML defines the structure of a web page, **CSS** controls its presentation and **JavaScript** adds client-side interaction [4]–[6]. **Bootstrap** provides a responsive grid and ready-made components that speed up the development of layouts that adapt to different screen sizes [3].

Django is a high-level Python web framework that follows the Model–Template–View (MTV) pattern. It provides URL routing, templates, forms, an authentication system with password hashing and sessions, an administration interface and an object-relational mapper (ORM) [1], [9]. Using the ORM, database tables are described as Python classes, which reduces handwritten SQL and the risk of SQL-injection errors.

MySQL is an open-source relational database management system that stores data in tables with primary and foreign keys and enforces integrity constraints [2], [10]. A relational model is a natural fit for sports data, where teams have players, players have attendance and performance records, and matches belong to teams.

Security guidance from OWASP stresses that authentication must use proven mechanisms and that authorization must be enforced on the server for every protected request [7], [8]. This guidance shapes the design decisions in Chapters 5 and 7.

2.4 Research Gap

For a small team, adopting a large commercial platform may be unnecessary, while using disconnected tools creates fragmented records. The gap addressed by this project is a simple, open-source, role-based system that offers the core administrative functions of a team in a single responsive application. The contribution is the integration of the selected functions with proper access control and a documented test plan; the system does not claim any novel sportsanalytics algorithm.

Table 2.1: Comparison of existing and proposed approaches

Approach	Advantages	Limitations	Role-based access
Paper registers	Low cost; easy to start	Hard to search, back up and share	No
Spreadsheets	Flexible; familiar	Version conflicts; easy to overwrite data	Very limited
Messaging groups	Fast communication	Schedules and records get buried in chat	No
Commercial platforms	Feature-rich; polished	Subscription cost; more than a small team needs	Yes
Proposed SportTrackOTM	Central records; browserbased; tailored to the team	Needs setup, hosting and user training	Yes

Chapter 3 Problem Statement

3.1 Problem Definition

To design and develop a responsive web application that centralizes sports team administration by supporting role-based user access, team and player records, match and training schedules, attendance, performance information, results and announcements.

The system must be usable on common browsers and devices, must protect data through authentication and server-side permission checks, and must keep every record consistent in a relational database.

3.2 Challenges and Proposed Solutions

The main practical challenges and the design decisions taken to address them are summarised in Table 3.1.

Table 3.1: Challenges in team administration and the proposed solutions

Challenge	Effect	Proposed solution
Scattered information	Confusion and duplicate records	Centralized relational database with foreignkey relationships
Missed schedule updates	Players miss matches or practice	Schedule views and announcements on every dashboard
Manual attendance tracking	Time-consuming summaries	Digital attendance records with automatic percentages
Limited performance history	Difficult to review progress	Player-wise performance history linked to matches
Unauthorized changes	Incorrect or exposed information	Authentication and server-side role-based permissions
Small-screen usability	Poor access on phones	Responsive HTML/CSS/Bootstrap interface

3.3 Assumptions and Constraints

- Users have a modern browser and an internet or local-network connection.
- An administrator creates or approves accounts and assigns roles.
- Coaches enter attendance, results and performance information accurately.
- The first version is a web application, not a native Android or iOS application.
- The scope and schedule are suitable for a student project; advanced integrations are deferred.

3.4 Expected Outcomes

The expected outcome is a working web application with separate administrator, coach and player experiences, validated forms, persistent records and a testable workflow for managing team activities. Screenshots and measured test results will be added after the application is implemented and tested; none are claimed in this report.

Chapter 4 Project Requirements

This chapter lists the hardware, software, functional and non-functional requirements of the system. The tools named are representative choices; equivalent alternatives can be substituted without changing the architecture.

4.1 Hardware Requirements

Table 4.1: Hardware requirements

Component	Minimum specification / purpose
Development computer	Laptop or desktop able to run Python, Django, MySQL and a browser
Memory and storage	Sufficient RAM and free disk space for the IDE, dependencies and the database
Network	Internet for package installation and optional deployment; local network for local testing
Test devices	A desktop browser and at least one mobile-sized viewport for responsive checks

4.2 Software Requirements

Table 4.2: Software requirements

Category	Proposed tool
Operating system	Windows, Linux or macOS
Editor / IDE	Visual Studio Code
Backend	Python 3 and Django
Frontend	HTML5, CSS3, Bootstrap, JavaScript
Database	MySQL (with the mysqlclient driver)
Version control	Git and GitHub (recommended)
Testing	Django test framework and browser-based manual testing

4.3 Functional Requirements

Table 4.3: Functional requirements

ID	Requirement
FR1	The system shall authenticate registered users and support logout.
FR2	The system shall distinguish administrator, coach and player roles.
FR3	The administrator shall create and manage teams, users, coaches and players.
ID	**Requirement**
FR4	Coaches shall view the rosters of their assigned teams.

FR5	Authorized users shall create and view match schedules.
FR6	Coaches shall create and update training sessions.
FR7	Coaches shall record attendance for a training session.
FR8	Coaches shall record match results and player performance.
FR9	Authorized users shall publish team announcements.
FR10	Players shall view their own schedules, attendance, results and performance.
FR11	The system shall validate form data and preserve records in MySQL.
FR12	The system shall display role-appropriate dashboard summaries.

4.4 Non-Functional Requirements

Table 4.4: Non-functional requirements

Attribute	Requirement
Security	Django authentication, password hashing, server-side permission checks and protected settings
Usability	Clear navigation, readable labels and consistent forms
Responsiveness	Pages adapt to desktop, tablet and mobile screen sizes
Reliability	Inputs are validated; missing or invalid data is handled gracefully
Maintainability	Models, views, templates and static files are separated into app modules
Performance	Common pages load efficiently under expected student-project usage
Data integrity	Foreign keys and constraints prevent orphaned or duplicate records

4.5 Feasibility Study

Technical feasibility: Django and MySQL are mature technologies with extensive documentation and community support, and the team has working knowledge of Python and web development.

Economic feasibility: the entire stack is open source and can be developed on an existing computer, so no licence cost is involved.

Operational feasibility: users interact only through a browser and need minimal training; dashboards present the information each role needs.

Schedule feasibility: delivering the core modules first and postponing advanced features keeps the project manageable within one semester.

Chapter 5
System Analysis – Proposed Architecture / High-Level Design

5.1 Overall Architecture

SportTrackOTM follows a layered, three-tier web architecture (Figure 5.1). The browser renders pages built with HTML, CSS, Bootstrap and JavaScript. Django URL routing sends each request to a view, which first passes through authentication and role checks. Permitted requests use Django models and the ORM to read or write the MySQL database, and the response is returned as an HTML page.

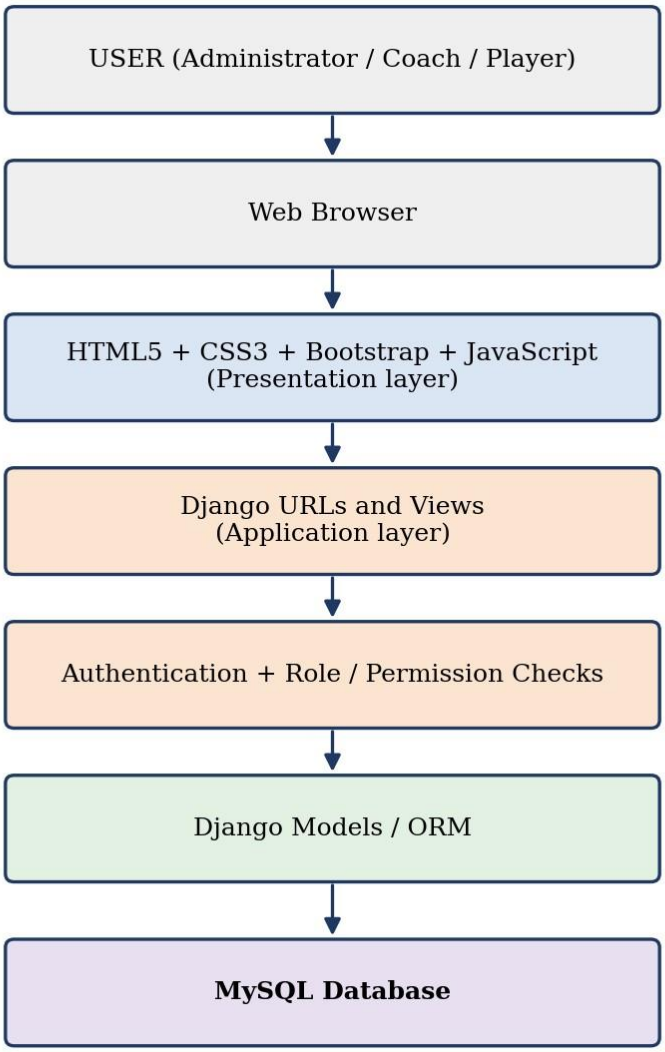


Figure 5.1: High-level architecture of SportTrackOTM (proposed).

5.2 Role-Based Workflow

The use-case view in Figure 5.2 shows the three actors and the functions available to each. Table 5.1 summarises the permissions. The same rules are enforced on the server for every request, not only by hiding buttons in the interface.

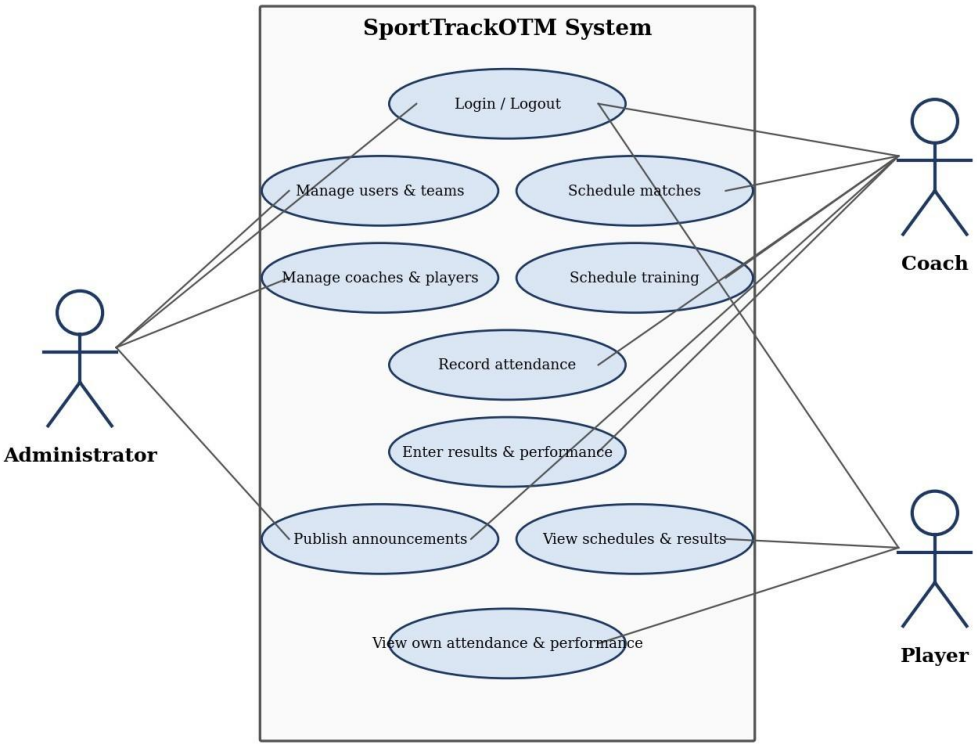


Figure 5.2: Use-case view of the role-based system.

Table 5.1: Role-wise permission matrix (planned)

Function	Administrator	Coach	Player
Login / logout	Yes	Yes	Yes
Manage users, coaches, players, teams	Yes	No	No
View team roster	All teams	Assigned teams	Own team
Schedule matches and training	Yes	Own team	No
Record attendance	No	Own team	No
Record results and performance	No	Own team	No
Publish announcements	General	Own team	No
View own attendance / performance	–	–	Yes
View schedules and results	All	Own team	Own team

5.3 Data Flow

Figure 5.3 shows the data flow. A user request first passes through authentication and authorization (process 1.0). Authorized users then manage teams, matches and training (2.0) or record attendance and performance (3.0), and announcements and reports are produced from

the stored data (4.0). Three data stores hold users and roles (D1), team and schedule data (D2), and attendance and performance data (D3).

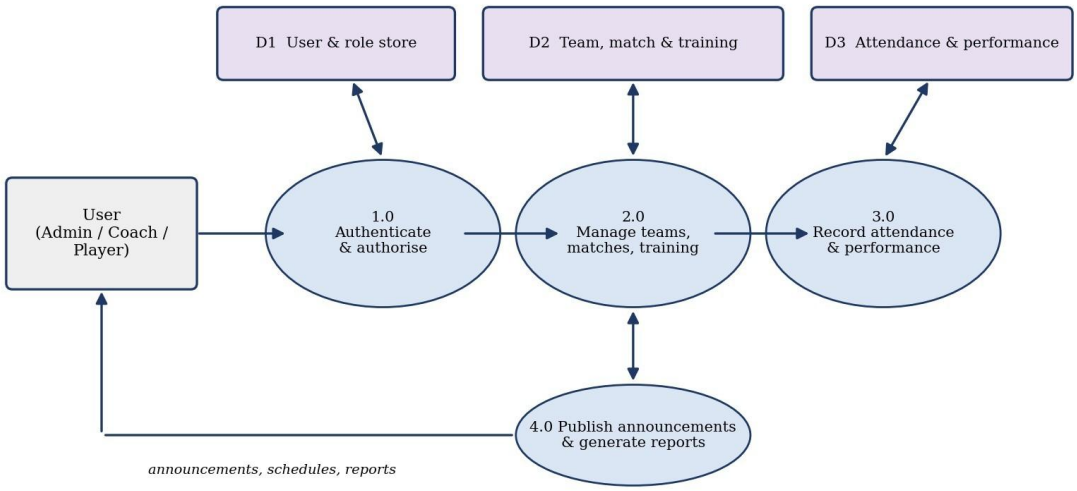


Figure 5.3: Data flow diagram of SportTrackOTM.

5.4 Module Description

Table 5.2: Main system modules

Module	Description
Authentication	Login, logout, session management and role-based access
User management	Create and maintain accounts and role assignments
Team and player management	Store team details, player profiles and team membership
Match management	Record opponent, date, time, venue, status and result
Training management	Publish practice sessions with location and time
Attendance	Record Present / Absent / Late status per player and session
Performance	Store player statistics and coach remarks for each match
Announcements	Publish team-specific or general updates
Dashboard and reports	Upcoming matches, recent results, attendance overview

5.5 Database Design

The proposed relational schema is shown as an entity-relationship overview in Figure 5.4 and described in Table 5.3. Foreign keys connect each record to its owner (for example, an attendance record to a player and a training session). Final field definitions and constraints will be confirmed during implementation.

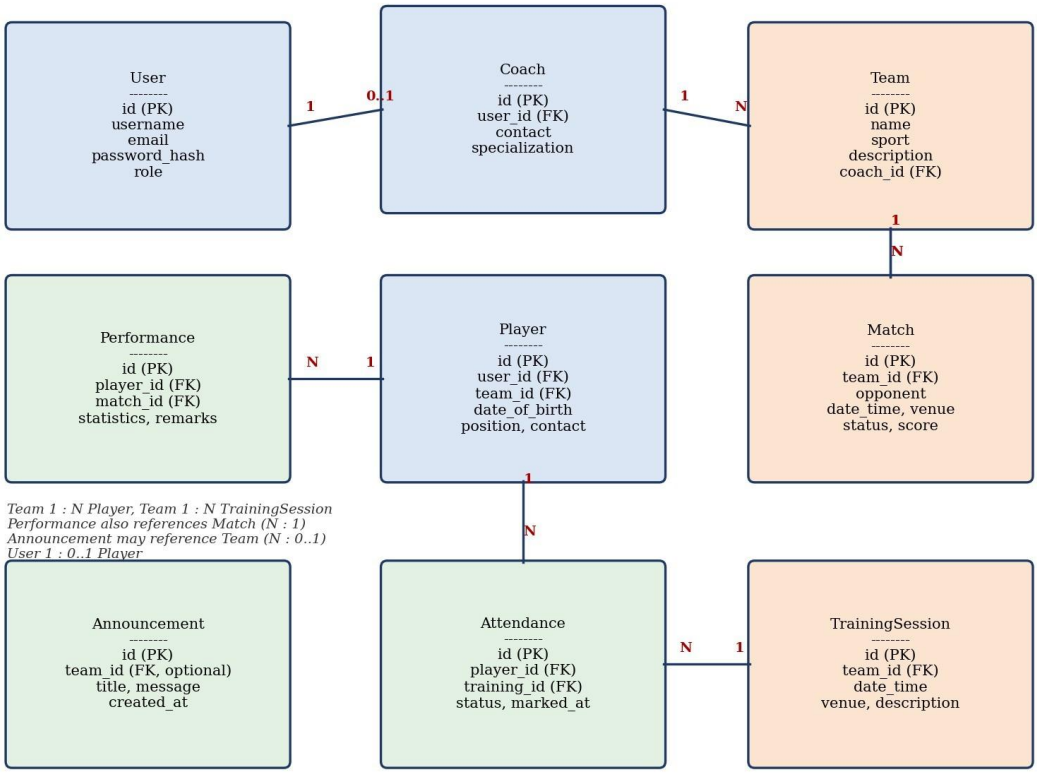


Figure 5.4: Proposed entity-relationship overview.

Table 5.3: Proposed database entities

Entity	Representative fields	Relationships
User	id, username, email, password hash, role	One account may map to a coach or player profile
Team	id, name, sport, description	Has players and an assigned coach
Player	id, user_id, team_id, date_of_birth, position, contact	Belongs to a team; has attendance and performance records
Coach	id, user_id, contact, specialization	May manage one or more teams
Match	id, team_id, opponent, date_time, venue, status, score	Belongs to a team; has performance records
TrainingSession	id, team_id, date_time, venue, description	Belongs to a team; has attendance records
Attendance	id, player_id, training_id, status, marked_at	Connects a player with a training session (unique pair)
Performance	id, player_id, match_id, statistics, remarks	Connects a player with a match
Announcement	id, team_id (optional), title, message, created_at	Targets one team or the whole system

5.6 Sequence of Operation

Figure 5.5 shows the sequence for any protected action. The user submits a request; Django checks the user's role; if access is allowed the view reads or writes the database through the ORM and returns an updated page, otherwise an error or redirect is returned.

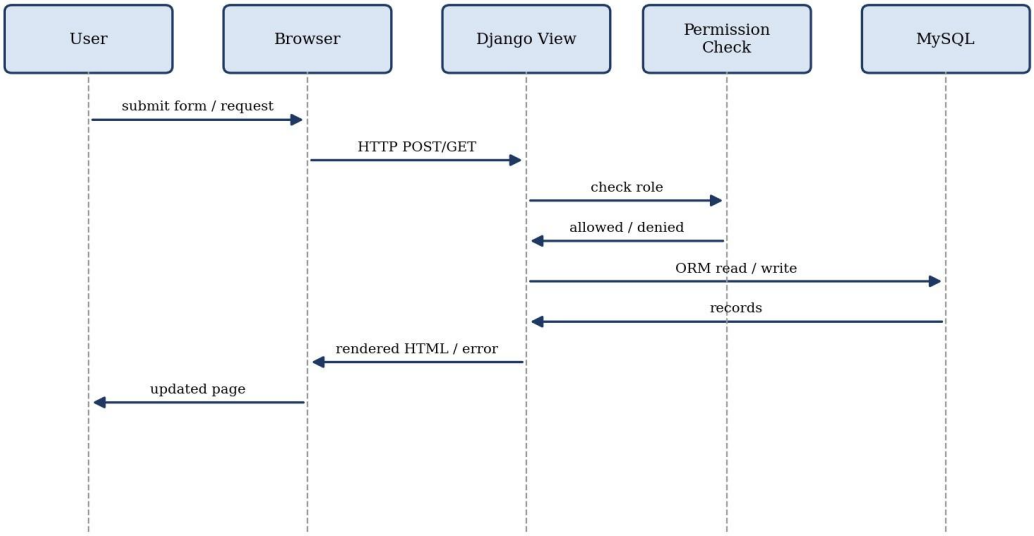


Figure 5.5: Sequence of a protected request.

5.7 Dashboard Design

- **Administrator dashboard:** counts of users and teams, recent activity and shortcuts to management pages.
- **Coach dashboard:** assigned teams, upcoming matches and training sessions, and quick links for attendance and performance entry.
- **Player dashboard:** next scheduled activity, recent results, personal attendance percentage, performance summary and latest announcements.

5.8 Design Considerations

- Use Django's built-in authentication; never store plain-text passwords.
- Check permissions on the server for every protected action; hiding a button is not sufficient security.
- Use foreign keys and appropriate deletion rules to preserve data integrity.
- Keep secrets and production credentials out of source control.
- Use responsive layouts and accessible labels for forms.
- Back up the database before major changes or deployment.

Chapter 6 Project Plan

6.1 Project Phases

The project is organised in phases that follow the development order of the system. Activities and deliverables are listed in Table 6.1 and the schedule is shown as a Gantt chart in Figure 6.1. Week numbers assume a 16-week semester and can be adjusted to the academic calendar.

Table 6.1: Project phases, activities and deliverables

Phase	Main activities	Deliverable	Weeks
1. Requirements	Confirm roles, features and scope	Requirements checklist	1–2
2. Design	Architecture, wireframes, database schema	Design and ER overview	2–4
3. Foundation	Django project, app structure, MySQL connection	Running base application	4–5
4. Authentication	Login/logout, roles, protected dashboards	Secure access flow	5–6
5. Core records	Teams, players, coaches (CRUD)	Record-management modules	6–8
6. Activities	Matches, training sessions, attendance	Scheduling and attendance workflow	8–10
7. Tracking	Results, performance, announcements	Activity and communication modules	10–12
8. Testing	Functional, validation, access and responsive checks	Test evidence and bug list	11–14
9. Finalization	Deployment, documentation, screenshots	Final report and demo	13–16

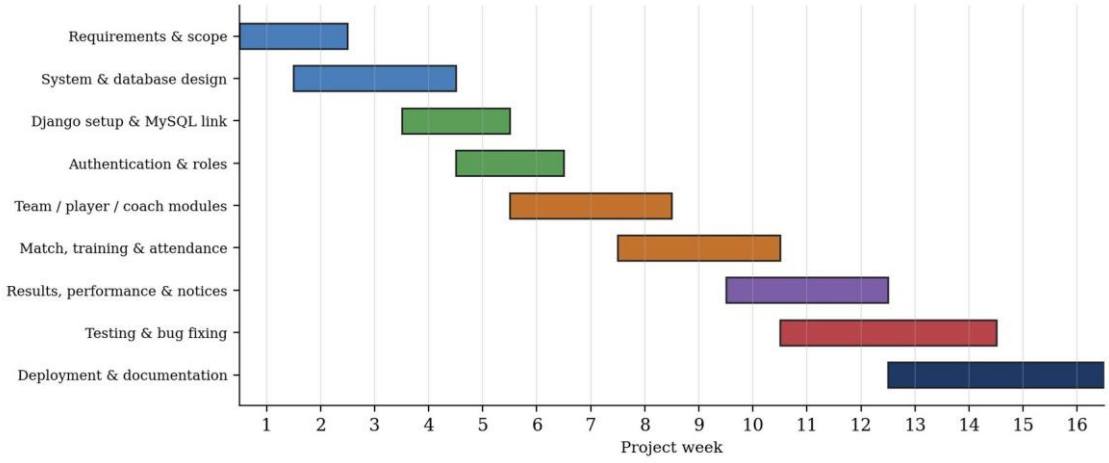


Figure 6.1: Proposed project schedule (16 weeks).

6.2 Risk Analysis

Table 6.2: Project risks and mitigation

Risk	Impact	Mitigation
Database connection errors	Development delay	Test the MySQL connection early; document configuration
Scope becomes too large	Incomplete modules	Deliver core features first; defer advanced integrations
Incorrect role permissions	Data exposure or unwanted changes	Test each role and enforce permissions on the server
Data loss	Missing project records	Regular database backups and Git version control
Limited testing time	Undetected defects	Write test cases while implementing each module
Responsive layout problems	Poor mobile experience	Test pages at several viewport sizes

6.3 Resources and Tools

The work is carried out as an individual project under the internal guidance of Prof. Jaydeep Lokhande. Primary resources are a development computer, VS Code, Python, Django, MySQL, a web browser and the official documentation listed in the references. Git is used for tracking changes, and a simple task checklist tracks module completion. Progress is reviewed with the guide at the end of each phase.

Chapter 7 Implementation

This chapter describes the intended implementation approach. The code listings are short representative sketches of each module, not the complete source; exact files, screenshots and details will be updated to match the final code.

7.1 Implementation Environment

The environment is Python 3 with Django, MySQL, HTML5, CSS3, Bootstrap and JavaScript, edited in Visual Studio Code. Development and initial testing run locally on Django's development server, which is meant for testing only and must not be used as the production server. A production deployment would use a WSGI server such as Gunicorn behind a web server, with DEBUG disabled and secrets read from environment variables.

7.2 Project Structure

```
SportTrackOTM/   manage.py
  config/          # settings and root URLs
  core/            # landing pages and shared utilities   accounts/        # custom user model,
login, role checks   teams/           # teams, coaches, player profiles   activities/      #
matches and training sessions   tracking/        # attendance and performance
announcements/   # team and general announcements   templates/       # HTML
templates (base + per role)
  static/          # CSS, JavaScript, images
```

Listing 7.1: Proposed project structure.

The app names are a proposed organisation and may be adjusted to match the repository. Separating the apps keeps each module small and testable.

7.3 Database Configuration

MySQL is connected through Django's database settings. Credentials are read from environment variables so that they are never committed to source control. Before the first migration the project sets a custom user model, because changing the user model after migrations exist is difficult.

```
# config/settings.py (excerpt)
import os
AUTH_USER_MODEL = 'accounts.User'
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': os.environ.get('DB_NAME', 'sporttrack'),
        'USER': os.environ.get('DB_USER', 'root'),
        'PASSWORD': os.environ['DB_PASSWORD'],
        'HOST': os.environ.get('DB_HOST', '127.0.0.1'),
        'PORT': os.environ.get('DB_PORT', '3306'),
    }
}
LOGIN_URL = '/accounts/login/'
```

Listing 7.2: Database and user-model configuration (sketch).

7.4 Authentication and Roles

Django authentication verifies credentials, hashes passwords and manages sessions. Roles are stored in a field of the custom user model. A small decorator checks the role of the logged-in user before a view runs and denies access otherwise, so that every protected view applies the same rule.

```
# accounts/models.py
from django.contrib.auth.models import AbstractUser
from django.db import models  class
User(AbstractUser):     class
Role(models.TextChoices):
       ADMIN = 'ADMIN', 'Administrator'
       COACH = 'COACH', 'Coach'
       PLAYER = 'PLAYER', 'Player'
   role = models.CharField(max_length=10, choices=Role.choices,
                           default=Role.PLAYER)


# accounts/decorators.py from functools
import wraps
from django.contrib.auth.decorators import login_required from
django.core.exceptions import PermissionDenied  def role_required(*roles):     def
decorator(view):         @login_required
       @wraps(view)         def wrapper(request, *args, **kwargs):
if request.user.role not in roles:                 raise PermissionDenied
           return view(request, *args, **kwargs)
       return wrapper     return decorator
```

Listing 7.3: Custom user model and role decorator (sketch).

7.5 Team and Player Management

The administrator creates teams and assigns coaches. Each player profile is linked to a user account and, where applicable, to a team. Forms validate required fields and reject duplicate or invalid relationships.

```
# teams/models.py
from django.conf import settings from django.db import models  class
Team(models.Model):     name = models.CharField(max_length=100,
unique=True)
   sport = models.CharField(max_length=50)     description =
models.TextField(blank=True)
   coach = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
                        on_delete=models.SET_NULL,
related_name='coached_teams')  class Player(models.Model):     user =
models.OneToOneField(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE)
   team = models.ForeignKey(Team, null=True, on_delete=models.SET_NULL,
                       related_name='players')     date_of_birth = models.DateField(null=True,
blank=True)     position = models.CharField(max_length=50, blank=True)
```

Listing 7.4: Team and player models (sketch).

7.6 Match and Training Management

A match record stores the team, opponent, date and time, venue, status and result. A training session stores the team, schedule, venue and description. Validation rejects past dates for new fixtures, and clear status labels (Scheduled, Completed, Cancelled) help users tell upcoming and finished activities apart.

7.7 Attendance and Performance

Attendance links one player to one training session with a status of Present, Absent or Late. A uniqueness constraint prevents duplicate records for the same player and session, which also protects against double submission of the form. Performance records link a player to a match and store general statistics and coach remarks; sport-specific fields can be added later.

```
# tracking/models.py from django.db import
models from teams.models import Player
from activities.models import TrainingSession  class
Attendance(models.Model):
   STATUS = [('P', 'Present'), ('A', 'Absent'), ('L', 'Late')]     player = models.ForeignKey(Player,
on_delete=models.CASCADE)
   training = models.ForeignKey(TrainingSession, on_delete=models.CASCADE)
   status = models.CharField(max_length=1, choices=STATUS)     marked_at =
models.DateTimeField(auto_now_add=True)      class Meta:         constraints =
[models.UniqueConstraint(
         fields=['player', 'training'], name='one_record_per_session')]
```

Listing 7.5: Attendance model with a uniqueness constraint (sketch).

7.8 Announcements and Reports

Authorized users create announcements with a title, message, date and optional team scope. Dashboards show recent announcements and upcoming activities. Reports summarise attendance percentages, match outcomes and player statistics. Any figure in the final submission will be generated from actual application data.

Chapter 8 Performance Evaluation and Testing

8.1 Evaluation Strategy

Testing is carried out incrementally as each module is completed [11]. The plan combines functional tests, permission tests, form-validation checks, database-persistence checks and responsive-interface checks, using Django's test framework for automated tests and manual browser testing for the interface. No test is marked as passed until it has been executed and its outcome recorded.

8.2 Evaluation Metrics

- **Functional correctness:** proportion of executed test cases that pass.
- **Access control:** unauthorized actions are denied for each role.
- **Data integrity:** submitted records persist and relationships remain valid.
- **Usability:** common tasks can be completed with clear navigation and messages.
- **Responsiveness:** essential pages remain usable at desktop and mobile viewport sizes.
- **Performance:** page response time under a defined test setup, if measured.

8.3 Test Conditions

Table 8.1: Recommended test conditions

Condition	Purpose
Synthetic accounts for each role	Test each role separately without using real personal data
Valid and invalid inputs	Check validation of required fields and invalid dates
Duplicate attendance submission	Confirm the uniqueness constraint works
Access to another role's URL	Confirm server-side permission checks
Desktop and mobile viewports	Confirm the responsive layout
Logged-out access to protected pages	Confirm redirect to login

8.4 Functional Test Plan

The functional requirements of Chapter 4 are verified with the test cases in Table 8.2. The Status column is intentionally left as 'Not run' until each test has been executed.

Table 8.2: Functional test cases

ID	Test case	Expected result	Status
TC01	Submit valid login credentials	User reaches the dashboard for their role	Not run
TC02	Submit invalid login credentials	Login rejected with a safe error message	Not run

ID	Test case	Expected result	Status
TC03	Player opens an admin-only URL	Access denied or redirected	Not run
TC04	Admin creates a team	Team saved and shown in the team list	Not run
TC05	Coach creates a match	Match stored and shown in the schedule	Not run
TC06	Coach records training attendance	Attendance saved for the session	Not run
TC07	Coach records a match result	Result visible to authorized users	Not run
TC08	Player views own performance	Only that player's data is displayed	Not run
TC09	Submit a form with missing data	Validation message; record not saved	Not run
TC10	Open dashboard on a mobile viewport	Layout readable and usable	Not run
TC11	Publish an announcement	Appears to its intended audience	Not run
TC12	Log out and revisit a protected page	Unauthenticated access denied	Not run

8.5 Non-Functional Testing

Security testing verifies server-side permission checks and safe handling of credentials. Usability testing inspects labels, navigation, error messages and empty states. Compatibility testing uses current versions of major browsers. Performance observations are recorded only together with the hardware, dataset size and test procedure used.

Chapter 9 Results and Analysis

9.1 Current Development Status

At the time of writing, the project is at the planning and setup stage. The requirements, architecture, module breakdown, database entities, implementation approach and test plan are documented in this report. A completed implementation and measured results have not yet been verified, so none are reported.

9.2 Result Table

Table 9.1 is the template in which verified results will be recorded after implementation and testing.

Table 9.1: Result recording template

Feature	Status	Evidence to attach
Authentication and roles	Pending verification	Login screenshots and role tests
Team / player management	Pending verification	Create, edit and list test results
Match scheduling	Pending verification	Schedule and result screenshots
Training and attendance	Pending verification	Attendance records and test cases
Performance tracking	Pending verification	Player performance view
Announcements	Pending verification	Announcement page and visibility test
Responsive interface	Pending verification	Desktop and mobile screenshots

9.3 Analysis Framework

After testing, the report will summarise the number of test cases executed and passed, accesscontrol failures found, validation defects and response-time observations. Defects and their fixes will be explained rather than reporting only a pass percentage, and any performance claim will be supported by repeatable measurements.

9.4 Discussion

Expected advantages

- A single, shared source of team information instead of scattered files.
- Reduced administrative effort for coaches through digital attendance and scheduling.
- Role-based access that protects data and keeps each dashboard simple.
- Low cost, because the whole stack is open source.

Limitations

- Data quality depends on accurate and timely entry by coaches.
- The first version offers reliable administration, not advanced analytics or live integrations.
- Benefits will depend on implementation quality and adoption by users.
- Measured results are still to be reported.

Chapter 10 Applications

The system can be applied wherever a group needs to manage players, schedules, attendance and performance records. Table 10.1 lists typical areas.

Table 10.1: Application areas of SportTrackOTM

Area	Use
College and university teams	Shared roster, fixtures and practice schedule
Local clubs and amateur teams	Coordinating matches and training sessions
Coaches	Maintaining attendance and performance history
Sports coordinators	Publishing team-wide notices and schedule changes
Sports academies	Tracking many players across several teams
Student projects	Demonstrating role-based web development and relational design

The system can be adapted to different sports. Performance fields and terminology should be configured for each sport rather than assuming that one set of statistics fits every activity.

Chapter 11 Conclusions

SportTrackOTM is proposed as a responsive, web-based Online Sports Team Management System that centralizes user, team, match, training, attendance, performance, result and announcement information. The administrator, coach and player roles give a clear division of responsibilities, and Django with MySQL offers a practical foundation for authentication, business rules and structured records, while Bootstrap supports access from desktop and mobile browsers.

This report has defined the problem, scope, requirements, architecture, database entities, implementation approach and test plan. Because implementation and evaluation are still to be completed, the report does not claim measured performance or completed feature validation. The next steps are to finalise the user model and database configuration, implement authentication, and then build each module in the planned sequence, recording test results as the work progresses.

Chapter 12
Future Prospects of the Project

The following extensions are outside the initial scope and should be prioritised only after the core application has been implemented, tested and reviewed.

Table 12.1: Future work directions

Direction	Description
Progressive Web App	Install-like access and selected offline views
Notifications	Email or push reminders for matches and training
Calendar integration	Calendar export and sync with common calendar services
Analytics	Sport-specific statistics and visual dashboards
Exports	CSV / PDF export of attendance and match reports
Registration	Team invitations and controlled self-registration
Multi-team organisations	Multiple teams per organisation with finer-grained permissions
Mobile application	Native app using a REST API on the same backend
Scheduling assistance	Automated fixture suggestions subject to coach approval

References

[1] Django Software Foundation, “Django documentation,” https://docs.djangoproject.com/ (accessed 14 Oct. 2026).

[2] Oracle, “MySQL Reference Manual,” https://dev.mysql.com/doc/ (accessed 14 Oct. 2026).

[3] Bootstrap Team, “Bootstrap Documentation,” https://getbootstrap.com/docs/ (accessed 14 Oct. 2026).

[4] MDN Web Docs, “HTML: HyperText Markup Language,” https://developer.mozilla.org/docs/Web/HTML (accessed 14 Oct. 2026).

[5] MDN Web Docs, “CSS: Cascading Style Sheets,” https://developer.mozilla.org/docs/Web/CSS (accessed 14 Oct. 2026).

[6] MDN Web Docs, “JavaScript,” https://developer.mozilla.org/docs/Web/JavaScript (accessed 14 Oct. 2026).

[7] OWASP Foundation, “Authentication Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (accessed 14 Oct. 2026).

[8] OWASP Foundation, “Authorization Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html (accessed 14 Oct. 2026).

[9] A. Holovaty and J. Kaplan-Moss, The Definitive Guide to Django: Web Development Done Right, 2nd ed. Berkeley, CA, USA: Apress, 2009.

[10] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[11] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2016.